🔧 Functions in C Programming

Building Blocks of Modular Programming

📝 Definition 🔄 Calling 🧠 Recursion

Introduction

🔧 To make programming **simple and easy to debug**, we break a larger program into smaller subprograms which perform 'well-defined tasks'.

Benefits of Functions

- ✓ Reduces complexity
- ✓ Increases reusability
- ✓ Makes debugging easier
- ✓ Enables modular programming

💡 **Key Concept:** Functions are C building blocks where every program activity occurs. It is a self-contained program segment that carries out a specific, well-defined task.

Definition of a Function

A function is a **group of statements** that together perform a task.

Key Components

- ✓ Function name
- ✓ Return type
- ✓ Parameter list
- ✓ Function body
- ✓ Return statement

```
return_type function_name(parameter_list) {
    local variable declarations;
    executable statement 1;
    executable statement 2;
    ---------------------------
    ---------------------------
    executable statement n;
    return (expression);
}
```

Function Categories

Functions in C can be classified into two main categories:

Library Functions

Predefined in standard library of C. Need is just to include appropriate library.

```
#include
#include
#include
```

Examples: printf(), scanf(), strlen(), strcpy(), etc.

User-Defined Functions

It has to be developed by the user at the time of program writing.

```
int max(int num1, int num2);
double average(int arr[], int size);
```

Functions that solve specific problems for your program.

🔧 Need for User-Defined Functions

🔧 If a program is divided into functional parts, then each part may be independently coded and later combined into a single unit.

Advantages of Modular Approach

- ✓ Length of program can be reduced by using functions
- ✓ Reusability of function increases
- ✓ It is easy to use
- ✓ Debugging is more suitable (easier) for programs
- ✓ It is easy to understand the actual logic of a program
- ✓ Highly suited in case of large programs
- ✓ By using functions in a program, it is possible to construct modular and structured programs

💡 **Key Point:** Functions are C building blocks where every program activity occurs. It is a self-contained program segment that carries out a specific, well-defined task.

📝 Declaration of a Function

📝 Before defining a function, it is appropriate to declare a function along with its prototype.

General Form

```
([]);
```

A function declaration tells the compiler about a function's name, return type, and parameters.

- **Parameter Names**
  Only their type is required, not their names.

- **Global Declaration**
  Required when you define a function in one source file and call it in another.

💡 **Best Practice:** Declare functions at the top of the file or include them in a header file for better code organization.

🔄 Calling a Function

🔄 The main() function can call up these functions from several different places within the program, to carry out the required processing.

Function Call Syntax

```
result = function_name(arguments);
```

The return value can be stored in a variable or used directly in expressions.

🔹 **Multiple Calls**
Functions can be called multiple times with different arguments, producing different results.

🔹 **Control Transfer**
When a function call is made, program control is transferred to the called function.

💡 **Key Point:** A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

📋 Function Arguments

📋 If a function is to use arguments, it must declare variables that accept values of arguments.

Formal Parameters

Behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

- **Scope**
  Limited to the function body

- **Lifetime**
  Created when function is called, destroyed when function returns

💡 **Key Point:** Changes made to formal parameters inside a function have no effect on the arguments used to call the function.

Return Statement

Information is returned from the function to the calling portion of the program via return statement.

Return Forms

- ✓ return;
- ✓ return (expression);
- ✓ or

```
int max(int num1, int num2) {
    int result;
    if (num1 > num2)
        result = num1;
    else
        result = num2;
    return result;
}
```

💡 **Key Point:** A function can have multiple return statements, each containing different expression.

💻 Example: max() Function

💻 Function that returns the maximum of two numbers.

```
/* function returning max between two numbers */
int max(int num1, int num2) {
    /* local variable declaration */
    int result;

    if (num1 > num2)
        result = num1;
    else
        result = num2;

    return result;
}
```

Function Prototype

💡 **Example Usage:**

```
#include
/* function declaration */
int max(int num1, int num2);
int main() {
   /* local variable definition */
   int a = 100;
   int b = 200;
   int ret;

   /* calling a function to get max value */
   ret = max(a, b);

   /* output the returned value */
   printf("Max value is : %d\n", ret);

   return 0;
}
```

For the above defined function max(), the function declaration is as follows:

```
int max(int num1, int num2);
```

Parameter names are not important in function declaration, only their type is required.

Output: Max value is : 200

🎯 Summary

Definition

Group of statements that perform a specific task

Calling

main() function calls other functions

Prototyping

Declares functions before use

🚀 Functions are building blocks of modular, structured C programming!